

Projeto Projeto Interdisciplinar – Aplicativo Mobile para a Próxima Etapa	Curso Análise e Desenvolvimento de Sistemas	Turma 3º Semestre 2026
Tema Aplicativo Android integrado a dados educacionais para acompanhamento da jornada do aluno	Título Próxima Etapa – App do Aluno	

PRÓXIMA ETAPA – APP DO ALUNO

Versão: FINAL

Aplicativo Mobile Android para Cursos, Presença, Certificados e Orientação Profissional

Projeto Interdisciplinar – 3º ADS– 2º SEMESTRE 2026

1. Apresentação do Projeto

A **Próxima Etapa** é uma organização sem fins lucrativos cuja missão é democratizar o acesso à educação superior e à orientação profissional para jovens de escolas públicas. Mais informações institucionais podem ser consultadas em proximaetapa.org.br.

Atualmente, a organização dispõe de um website e mantém informações relacionadas aos alunos participantes, cursos, atividades, cards, certificados e demais ações desenvolvidas junto aos jovens atendidos.

Com o objetivo de aprimorar a experiência dos estudantes e ampliar o relacionamento entre a ONG e seus participantes, propõe-se às equipes do 3º semestre do curso de Análise e Desenvolvimento de Sistemas o desenvolvimento de um aplicativo mobile para dispositivos Android, integrado a uma API e ao banco de dados disponibilizado pela Próxima Etapa.

O aplicativo deverá funcionar como um canal direto entre a Próxima Etapa e seus estudantes, oferecendo acesso simplificado às informações e serviços relacionados à jornada do participante.

Funcionalidades centrais previstas:

- recebimento de mensagens e informações sobre novos cursos;
- consulta à agenda de cursos e atividades;
- registro de presença por QR Code;
- acesso ao card digital do aluno;
- consulta e download de certificados;
- realização de testes de perfil para orientação educacional/profissional;
- chat ou canal de dúvidas dos alunos.

Para fins acadêmicos, as equipes deverão utilizar o banco de dados fornecido pela ONG para simular a integração entre o aplicativo desenvolvido e o website existente, contemplando cadastros de alunos, cursos, cards, certificados e demais informações necessárias. Não será necessário substituir o site institucional: deverá ser construído um módulo web administrativo e/ou uma camada de integração que represente as funções necessárias para alimentar o aplicativo.

2. Objetivos

2.1 Objetivo Geral

Desenvolver uma solução tecnológica composta por aplicativo Android, API REST e banco de dados, proporcionando aos alunos atendidos pela Próxima Etapa uma experiência digital integrada para acompanhamento de cursos, agenda, presença, certificados, perfil e comunicação com a organização.

2.2 Objetivos Específicos

- Desenvolver um aplicativo Android centrado na experiência do aluno participante.
- Integrar o aplicativo ao banco de dados disponibilizado pela Próxima Etapa por meio de uma API.
- Permitir ao estudante consultar cursos e atividades nos quais está inscrito e sua agenda individual.
- Implementar o controle de presença utilizando QR Code.
- Disponibilizar card digital e certificados do estudante.
- Criar testes de perfil baseados em perguntas e critérios de classificação previamente cadastrados.
- Implementar mensagens, notificações e um canal de comunicação para esclarecimento de dúvidas.
- Aplicar técnicas de análise descritiva sobre os dados disponibilizados ou simulados.
- Aplicar conceitos de orientação a objetos e estruturas de dados em uma solução integrada.
- Aplicar vocabulário técnico e comunicação profissional em língua inglesa.
- Desenvolver a solução observando segurança, acessibilidade, privacidade e princípios da LGPD.

3. Escopo e Regras de Negócio

A solução será organizada em dois módulos principais:

- Aplicativo Mobile – Módulo Aluno: utilizado pelos jovens participantes da Próxima Etapa.
- Cada estudante deverá possuir uma conta individual e visualizar somente os próprios dados autorizados.
- Cursos poderão possuir diferentes datas, horários, locais, carga horária e quantidade de encontros.
- O sistema deverá manter o relacionamento entre alunos, cursos, atividades, inscrições e presenças.
- Uma presença deverá estar relacionada ao aluno, curso/atividade, data e horário, evitando duplicidades para o mesmo encontro.
- O certificado deverá ser disponibilizado quando o estudante atender aos critérios definidos pela Próxima Etapa, registrado no banco de dados.
- Os testes de perfil deverão possuir perguntas, alternativas, critérios de pontuação/classificação e resultado, de acordo com o banco de dados.
- Os resultados dos testes de perfil terão finalidade de orientação educacional e profissional e não deverão ser apresentados como diagnóstico psicológico ou clínico.
- Mensagens e notificações poderão ser destinadas a todos os alunos ou a grupos específicos.
- A API deverá funcionar como camada de comunicação entre aplicativo e banco de dados.
- O banco disponibilizado pela ONG deverá ser preservado. Quando necessário, a equipe poderá implementar uma camada de adaptação ou cópia de desenvolvimento da estrutura original.
- Dados reais deverão ser utilizados somente quando autorizados. Para testes e apresentações, deverão ser priorizados dados fictícios, anonimizados ou previamente preparados.

4. Requisitos Funcionais – Funcionalidades Esperadas

4.1 Aplicativo Mobile – Módulo Aluno

ID	Funcionalidade	Descrição mínima
RFM01	Autenticação	Permitir login e logout do aluno e manter sessão autenticada com segurança.
RFM02	Página inicial	Apresentar resumo da jornada do aluno: próximos cursos, atividades, mensagens e atalhos.
RFM03	Cursos	Consultar cursos disponíveis, cursos inscritos e informações como descrição, período, carga horária e local.
RFM04	Novos cursos	Informar ao estudante quando novos cursos ou oportunidades forem disponibilizados.
RFM05	Agenda	Exibir agenda individual com cursos, encontros e demais atividades programadas.
RFM06	QR Code – Presença	Permitir registrar presença em atividades por meio de QR Code.
RFM07	Histórico de presença	Permitir ao estudante consultar presenças e participação nos cursos.
RFM08	Card do aluno	Exibir o card digital do participante com as informações autorizadas pela organização.
RFM09	Certificados	Listar certificados obtidos e permitir sua visualização e/ou download.
RFM10	Testes de perfil	Permitir ao aluno responder questionários e visualizar sua classificação ou resultado.
RFM11	Mensagens	Disponibilizar mensagens e comunicados enviados pela Próxima Etapa.
RFM12	Chat	Disponibilizar um canal para esclarecimento de dúvidas dos estudantes.
RFM13	Notificações	Enviar notificações relacionadas a novos cursos, alterações de agenda, certificados e informações relevantes.
RFM14	Perfil	Permitir ao estudante consultar e, quando autorizado, atualizar determinadas informações pessoais.
RFM15	Sincronização	Sincronizar informações locais do aplicativo com a API e o banco de dados, tratando falhas de conexão.

5. Requisitos Não Funcionais

5.1 Aplicativo Mobile

- Plataforma: funcionamento em smartphones Android.
- Usabilidade: interface simples, consistente e adequada ao público jovem.
- Acessibilidade: contraste adequado, textos legíveis, rótulos compreensíveis e suporte às boas práticas de acessibilidade.
- Desempenho: consultas e listagens com resposta adequada mesmo com crescimento do volume de dados.
- Persistência local: informações essenciais mantidas localmente quando necessário.
- Sincronização: tratamento de indisponibilidade temporária da internet e atualização posterior.
- Segurança: proteção das credenciais, comunicação HTTPS e armazenamento seguro de tokens/sessões.
- Privacidade: dados apresentados deverão ser restritos ao estudante autenticado.
- Manutenibilidade: organização adequada de classes, componentes e responsabilidades.
- Tratamento de erros: mensagens compreensíveis ao usuário diante de falhas.

5.2 API e Banco de Dados

- API REST documentada e com validação das informações recebidas.
- Consultas parametrizadas para reduzir riscos de SQL Injection.
- Uso adequado de chaves, restrições e transações para preservar a integridade dos dados.
- Registro de logs e versionamento do código-fonte.
- Separação de responsabilidades em camadas e configuração por variáveis de ambiente.
- Banco de dados em ambiente acessível para demonstração, com estratégia básica de backup e recuperação.
- LGPD: coleta mínima de dados, controle de acesso e proibição de dados pessoais reais em repositórios, banners ou demonstrações não autorizadas.

6. Tecnologias Sugeridas

As equipes poderão adotar uma stack **coerente com os conteúdos trabalhados nas Unidades Curriculares e aprovada pelos professores**. Sugestões:

- Aplicativo Android: Java, Android Studio, Android SDK, XML/ConstraintLayout, Activities, Fragments, Intents e RecyclerView.
- Persistência e integração mobile: SQLite ou Room; Retrofit ou biblioteca equivalente; JSON.
- QR Code: ZXing, Google ML Kit ou solução equivalente.
- Notificações: Firebase Cloud Messaging ou mecanismo equivalente, quando aplicável.
- Módulo Web: HTML5, CSS3, JavaScript/TypeScript; React, Vue ou framework equivalente.
- Backend/API: Node.js + Express, Java + Spring Boot, .NET Web API ou tecnologia equivalente aprovada.
- Banco de Dados: preferencialmente a tecnologia/estrutura disponibilizada pela Próxima Etapa; MySQL, PostgreSQL ou SQLite para ambientes de desenvolvimento, quando autorizado.
- Análise de Dados: Microsoft Excel, Python com Pandas/Matplotlib ou ferramenta equivalente.
- Versionamento e gestão: GitHub e quadro de tarefas/backlog.
- Documentação e testes: OpenAPI/Swagger, Postman/Insomnia e testes automatizados quando aplicável.
- Design e prototipação: Figma.
- Deploy: plataforma de nuvem escolhida pela equipe, com variáveis de ambiente e instruções reproduzíveis.

7. Etapas do Projeto

1. Descoberta: compreender a Próxima Etapa, seus usuários, serviços, processos e a estrutura do banco de dados.
2. Definição do MVP: estabelecer funcionalidades prioritárias, backlog, responsabilidades e critérios de aceitação.
3. Modelagem: elaborar fluxo de navegação, protótipos, classes, estruturas de dados, arquitetura, mapeamento da base e contratos da API.
4. Fundação técnica: implementar banco de dados, API, autenticação e comunicação inicial aplicativo-servidor.
5. Aplicativo Mobile inicial: desenvolver as primeiras telas e o fluxo Login → Cursos → Agenda.
6. Integração: integrar o aplicativo à API e implementar QR Code, presença, armazenamento local, notificações e sincronização.
7. Recursos complementares: implementar card, certificados, teste de perfil, mensagens e chat.
8. Análise de Dados: produzir indicadores e análises sobre os dados disponibilizados ou simulados.
9. Testes e validação: executar testes funcionais, de integração, persistência, usabilidade e segurança.
10. Implantação e apresentação: publicar servidor e banco, gerar APK/AAB, finalizar documentação, banner e pitch.

8. Possíveis Extensões

- Integração da agenda com calendários externos.
- Funcionamento offline ampliado.
- Gamificação por participação, badges e trilhas de aprendizagem.
- Carteira digital de certificados.
- Indicação de cursos de acordo com interesses e perfil do estudante.
- Dashboard de evolução e participação do aluno.
- Sistema de inscrição e lista de espera.
- Avaliação de cursos e pesquisa de satisfação.
- Chatbot apoiado em base de conhecimento da ONG, com validação e proteção de dados.
- Painel de indicadores de impacto social.
- Geração automatizada de certificados.
- Integração futura com o website oficial da Próxima Etapa.

9. Entregáveis Gerais e Critérios de Aceitação

- Repositório versionado com código-fonte, histórico de commits e instruções de execução.
- Documentação de requisitos, regras de negócio, arquitetura e critérios de aceitação.
- Aplicativo Android e arquivo APK/AAB.
- Backend/API
- Banco de dados e scripts necessários para criação/carga no ambiente de demonstração.
- Documentação da API e evidências de testes.
- Análise estatística prevista na Unidade Curricular de Análise Descritiva de Dados.
- Documentação e parte da comunicação técnica em Business English.
- Sistema implantado em ambiente acessível, utilizando dados fictícios ou autorizados para demonstração.
- Relatório final, banner e apresentação com pitch e demonstração.

10. Entregáveis Intermediários por Unidade Curricular

As datas, pesos e formatos finais deverão ser confirmados no cronograma oficial dos professores. A matriz abaixo estabelece o conteúdo mínimo esperado e a integração progressiva entre as Unidades Curriculares.

Unidade Curricular	Entrega 1 – Parcial (3,0 pontos) 25 de Setembro 23:59:56	Entrega 2 -Final (4,0 pontos) 06 de Novembro 23:59:56	Apresentação (3,0 pontos) Semana FECAP de Tecnologia 17,18,19,23 e 24 de Novembro
Programação Mobile	Requisitos: desenvolver e entregar uma aplicação Android contendo duas ou mais telas para o aluno da Próxima Etapa. Função principal da etapa: Visualização dos cursos e da agenda individual do estudante. A aplicação deverá utilizar múltiplas Activities, Intents, Fragments, ConstraintLayout, TextView, ImageView, Button e RecyclerView ou equivalente. Deve integrar ao menos um dado numérico, como carga horária, número de encontros ou percentual de participação. A função principal deve estar programada, comentada e funcionando.	Requisitos gerais: ampliar a Entrega 1 para consumir dados de uma API REST do projeto, integrando autenticação e armazenamento local. Nova função principal: Sincronização dos dados do estudante e registro de presença por QR Code com persistência local e integração com a API. Fluxo principal: Login → Cursos → Agenda → Dados da API → QR Code/Check-in → Persistência → Sincronização → Histórico. Implementar tratamento de JSON, SQLite/Room, notificações, histórico, perda de conexão, card, certificados, teste de perfil e mensagens/chat.	Formato de entrega: Demonstração (demo) + banner e pitch de até 4 minutos. Critérios: Criatividade, impacto social, qualidade técnica, clareza, métricas e rastreabilidade. Aplicação web funcional com os três módulos, publicada em ambiente acessível e acompanhada de instruções de execução.
Cloud Native	AUTOMAÇÃO COM SHELL SCRIPT EM UBUNTU/LINUX OBJETIVO Aplicar, na prática, os conceitos de sistema operacional e linha de comando Linux estudados no trimestre (processos, permissões, sistema de arquivos, usuários, agendamento de tarefas, redirecionamento e pipes), desenvolvendo um ou mais scripts .sh que resolvam um problema real ou simulado de administração de sistemas.	CONTAINERIZAÇÃO COM DOCKER OBJETIVO Empacotar e orquestrar, usando Docker, uma aplicação (idealmente evoluída a partir do tema da Parte 1), aplicando conceitos de isolamento, portabilidade, imagens, volumes e redes de containers. REQUISITOS TÉCNICOS MÍNIMOS	

Unidade Curricular	Entrega 1 – Parcial (3,0 pontos) 25 de Setembro 23:59:56	Entrega 2 -Final (4,0 pontos) 06 de Novembro 23:59:56	Apresentação (3,0 pontos) Semana FECAP de Tecnologia 17,18,19,23 e 24 de Novembro
	REQUISITOS TÉCNICOS MÍNIMOS - Pelo menos 1 script principal em Bash (<code>#!/bin/bash</code>), podendo ser dividido em módulos auxiliares. - Uso de variáveis, estruturas condicionais e de repetição. - Uso de pelo menos 3 comandos Linux distintos relevantes ao problema (ex.: <code>find</code>, <code>grep</code>, <code>awk</code>, <code>sed</code>, <code>du</code>, <code>df</code>, <code>chmod</code>, <code>chown</code>, <code>cron</code>, <code>tar</code>, <code>curl</code>, <code>ps</code>, <code>kill</code>, <code>rsync</code> etc.). - Tratamento básico de erros (verificação de código de retorno, validação de parâmetros/argumentos). - Geração de log de execução (arquivo de texto com data/hora e resultado das ações). - Script deve ser executável (<code>chmod +x</code>) e receber parâmetros via linha de comando quando fizer sentido. - README.md explicando: o problema resolvido, como executar, dependências, exemplos de uso e prints/saída esperada. SUGESTÕES DE APLICAÇÃO PRÁTICA (escolham 1 tema ou proponham outro com aprovação do professor) 1. Script de backup automatizado Compacta (<code>tar/gzip</code>) uma pasta de "dados da aplicação", envia para um diretório de destino (local ou remoto via <code>rsync/scp</code>), mantém apenas as N cópias mais recentes e registra tudo em log. Pode ser agendado via <code>cron</code>. 2. Script de monitoramento de sistema Coleta uso de CPU, memória, disco e processos (via <code>top</code>, <code>free</code>, <code>df</code>, <code>ps</code>), gera um relatório periódico e dispara um alerta (e-mail simulado, mensagem no terminal ou arquivo de alerta) quando algum limite é ultrapassado. 3. Script de setup/provisionamento de ambiente Automatiza a instalação e configuração inicial de um ambiente de desenvolvimento (ex.: instala pacotes com <code>apt</code>, cria estrutura de pastas do projeto, configura usuário/permisões, variáveis de ambiente) - um "instalador" do próprio projeto integrado.	- Pelo menos 1 Dockerfile próprio (não usar apenas imagem pronta sem customização), com boas práticas básicas (imagem base enxuta, <code>dockerignore</code>, camadas otimizadas). - Pelo menos 2 serviços/containers integrados via <code>docker-compose.yml</code> (ex.: aplicação + banco de dados, ou aplicação + serviço de automação). - Uso de volume(s) para persistência de dados. - Uso de rede customizada ou comunicação explícita entre containers via nome de serviço. - Uso de variáveis de ambiente para configuração (arquivo <code>.env</code>, sem credenciais expostas no código). - README.md explicando: arquitetura dos containers (pode incluir um diagrama simples), como buildar e rodar (<code>docker compose up</code>), portas expostas e como testar. SUGESTÕES DE APLICAÇÃO PRÁTICA 1. Containerizar o projeto da Parte 1 Transformar o script de backup/monitoramento em um serviço que roda dentro de um container (ex.: com <code>cron</code> interno) e que faz backup de um banco de dados também containerizado (ex.: PostgreSQL/MySQL em outro container, com volume persistente). 2. Aplicação web + banco de dados Uma API simples (Node, Python/Flask, PHP etc.) em um container, conectada a um container de banco de dados, com um terceiro container opcional de administração (ex.: pgAdmin, phpMyAdmin). 3. Ambiente de desenvolvimento reproduzível Empacotar uma ferramenta ou aplicação legada/desktop como serviço web em container, permitindo que qualquer colega "suba o ambiente" com um único comando. 4. Micro-arquitetura simples de 3 camadas Front-end estático (Nginx), back-end (API) e banco de dados, cada um em seu container, comunicando-se via rede Docker.	

Unidade Curricular	Entrega 1 – Parcial (3,0 pontos) 25 de Setembro 23:59:56	Entrega 2 -Final (4,0 pontos) 06 de Novembro 23:59:56	Apresentação (3,0 pontos) Semana FECAP de Tecnologia 17,18,19,23 e 24 de Novembro
	4. Script de gerenciamento de usuários e permissões Cria usuários e grupos em lote a partir de um arquivo CSV, define permissões em diretórios específicos e gera relatório de auditoria de acessos. 5. Script de análise/limpeza de logs Varre logs de uma aplicação (reais ou simulados), filtra erros com grep/awk, gera estatísticas (quantidade de erros por tipo, por hora) e arquiva/roda logs antigos (log rotation manual). 6. Script de deploy simples Automatiza o processo de "colocar em produção" uma aplicação simples (ex.: puxa código do Git, para o serviço, copia arquivos, reinicia o serviço, valida se subiu corretamente).	5. Pipeline de automação containerizada Um container que executa periodicamente uma tarefa (ex.: coleta de dados, processamento de arquivo, envio de relatório), evoluindo o script Shell da Parte 1 para dentro de um container agendado (cron no container ou docker compose com serviço recorrente).	
Testes e Qualidade de Software (DEVOPS)	Instruções de Entrega: Qualidade de Software Objetivo: O aluno deverá demonstrar o entendimento dos conceitos de qualidade de software, aplicando-os ao projeto. Isso inclui a criação de um diagrama do processo de qualidade de software, a identificação de atributos de qualidade. Apresentar um relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento.	Instruções de Entrega: Testes Unitários e de Integração Objetivo: Os alunos deverão desenvolver 4 testes unitários e 2 testes de integração para o sistema desenvolvido. Esses testes devem garantir que o sistema esteja funcionando corretamente e validando os principais componentes e integrações e a geração de um relatório de teste de carga do servidor. Teste de Sistema / Aceitação Objetivo: O aluno deverá criar um teste de sistema ou aceitação para verificar se o sistema como um todo está funcionando conforme o esperado. O teste de aceitação verifica se as funcionalidades implementadas atendem aos requisitos do cliente ou usuário final, garantindo que o sistema está pronto para ser utilizado em produção.	
UX	Cada grupo aplica o processo de UX diretamente sobre o app do aluno da Próxima Etapa, evoluindo do wireframe ao mockup final das telas principais (login, cursos, agenda, presença, card digital, certificados, teste de perfil).	Cada grupo aplica o processo de UX diretamente sobre o app do aluno da Próxima Etapa, evoluindo do wireframe ao mockup final das telas principais (login, cursos, agenda, presença, card digital, certificados, teste de perfil).	

Unidade Curricular	Entrega 1 – Parcial (3,0 pontos) 25 de Setembro 23:59:56	Entrega 2 -Final (4,0 pontos) 06 de Novembro 23:59:56	Apresentação (3,0 pontos) Semana FECAP de Tecnologia 17,18,19,23 e 24 de Novembro
	Formato: as duas entregas compõem um único PDF, atualizado ao longo do curso — a segunda soma o mockup ao wireframe já entregue, mostrando a evolução do projeto. O documento deve trazer, na capa, nome e RA de todos os integrantes do grupo, e cada tela deve estar identificada (nome da tela + etapa: wireframe ou mockup). 1ª entrega — Wireframe Reúne o que foi construído desde o início do curso: fundamentos de UX e IHC, heurísticas de usabilidade, leitura crítica do briefing real da Próxima Etapa com brainstorming (SCAMPER), pesquisa com usuários, persona, jornada do usuário, o processo do Duplo Diamante e arquitetura da informação. Com base nisso, cada grupo entrega o wireframe de baixa fidelidade das telas principais do app, mostrando estrutura, hierarquia e fluxo de navegação — sem cor, tipografia ou imagens.	Formato: as duas entregas compõem um único PDF, atualizado ao longo do curso — a segunda soma o mockup ao wireframe já entregue, mostrando a evolução do projeto. O documento deve trazer, na capa, nome e RA de todos os integrantes do grupo, e cada tela deve estar identificada (nome da tela + etapa: wireframe ou mockup). 2ª entrega: Mockup completo Incorpora o aprofundamento técnico em wireframing e prototipação, storytelling, avaliação heurística, testes de usabilidade e acessibilidade. Cada grupo entrega o mockup completo das mesmas telas, em alta fidelidade, com cores, tipografia, componentes visuais e identidade do projeto definidos, evoluindo diretamente do wireframe entregue na primeira etapa.	

11. Cronograma Sugerido de Atividades

Etapa	Período sugerido	Atividades e resultados esperados
1. Descoberta e planejamento	Semanas 1–2	Entendimento da ONG, definição dos usuários, análise da base de dados, escopo do MVP, backlog, repositório e critérios de aceitação.
2. Modelagem	Semanas 3–4	Arquitetura inicial, classes de domínio, estruturas de dados, contratos da API, fluxos, wireframes e protótipos.
3. Fundação técnica	Semanas 5–6	Autenticação, banco de dados, API inicial e comunicação básica entre aplicativo e servidor.
4. Mobile inicial	Semanas 7–8	Login, cursos, agenda e primeira entrega funcional do aplicativo Android.
5. Integração e persistência	Semanas 9–10	Consumo da API REST, JSON, persistência local, tratamento de conexão e sincronização.
6. Presença e notificações	Semanas 11–12	QR Code, check-in, histórico de presença e notificações.
7. Recursos complementares	Semanas 13–14	Card, certificados, teste de perfil, mensagens/chat, análises e ajustes de UX.
8. Finalização	Semanas 15–16	Testes, deploy, documentação, APK/AAB, relatório, banner, ensaio do pitch e apresentação final.

Cronograma de Atividades/Tarefas

ATIVIDADE/TAREFA	NOTA	PENALIDADE	DATA LIMITE
Cadastro/formação do time (grupos definitivos; regras de composição)	—	Cadastros indevidos/fora das regras: -1,0 ponto do grupo.	14 de Agosto
Aceitar o convite do repositório no GitHub (ver e-mail/SPAM)	—	Convite expira; se expirar e precisar reenvio, pode gerar desconto (ver abaixo).	Até 7 dias após o envio do convite
Solicitar reenvio do convite (se o convite expirar)	—	-1,0 ponto na nota final por cada reenvio; se pedir reenvio com semanas de atraso: -0,25 ponto por semana adicional.	Após expirar (quanto antes, melhor)
Entrega 1 (E1) via GitHub/GitHub Project	3,0 pts	Atraso: perda de 25% da nota total por dia.	25/09 (até 23:59:56)
Enviar o Banner no GitHub para aprovação (pasta 'DOCUMENTOS')	—	Podemos reduzir até 25% da nota da apresentação para banners não aprovados.	Até 02/11
Avaliação 360° (entre pares)	(multiplicador 0,0 a 1,0)	Pode reduzir a nota final (fator multiplicador definido pela avaliação entre pares).	Entre 07/11 e 08/11
Entrega 2 + Documento de extensão COM EMPRESA - Final (E2) via GitHub/GitHub Project	4,0 pts	Atraso: perda de 25% da nota total por dia.	06/11 (até 23:59:56)
Enviar o Banner para a gráfica (TJ Impressos) (PDF + comprovante PIX)	—	Envio após o prazo: aumento de valor e risco de não produção a tempo.	Prazo máximo: 09/11
Apresentação (Pitch) + Demonstração do produto desenvolvido durante o semestre + Banner	3,0 pts	Sem banner: pode reduzir em até 25% a nota da apresentação.	Semana FECAP de Tecnologia 17,18,19,23 e 24 de Novembro